

AGASTYA — Final Routing Endpoint Regression Fix Report

SIH Problem Statement 26085: Physics-Guided Urban Flood Nowcasting & Emergency Routing

SIH PS 26085

Audit Status: 100% Remediated
Test Suite: 53/53 Passed (0 Failures)
Date: September 2026

Executive Summary

A rigorous, end-to-end regression remediation and verification was performed on the **Emergency Ambulance Routing Subsystem** of AGASTYA. The previous manual verification failure—where flooded origin or flooded destination endpoints allowed route computation or failed to render explicit operator warnings—has been completely resolved through:

- Canonical depth validation** preventing missing depths from silently defaulting to 0.0 cm.
- Pre-Dijkstra endpoint safety gates** halting computation and returning `ORIGIN_UNSAFE` and `DESTINATION_UNSAFE`.
- Immediate route invalidation** purging stale map polylines synchronously when rainfall, duration, or endpoints change.
- Multi-surface operator alerting:** High-priority top red banner, sidebar depth chips, and detailed route refusal status cards.

Verified across **53 automated pytest unit/regression tests**, a clean **Vite production build**, and live **browser automation verification**.

1. Root Cause Analysis & Codebase Remediations

Component Layer	Defect Identified	Exact Technical Remediation	Status
Backend Solver safe_route.py	Endpoint depth fallback: <code>depth_cm.get(source, 0.0)</code> silently considered missing nodes as 0.0 cm safe. Missing depths in response schema.	Enforced strict validation: <code>source, target in depth_cm</code> (returns <code>ENDPOINT_DEPTH_UNAVAILABLE</code>). Checked <code>source_depth > 15.0 cm</code> and <code>target_depth > 15.0 cm</code> before Dijkstra. Added <code>origin_depth_cm</code> and <code>destination_depth_cm</code> to all returns.	✓ FIXED
API & Schema Layer schemas.py, main.py	<code>RouteRequest</code> lacked client depth synchronization; <code>RouteResponse</code> schema dropped depth fields; 404 errors bypassed structured responses.	Added <code>depths</code> to <code>RouteRequest</code> ensuring single source of truth (<code>\$ ext{displayedDepths} \equiv ext{routingDepths}\$</code>). Added <code>origin_depth_cm</code> , <code>destination_depth_cm</code> , and <code>threshold_cm</code> to <code>RouteResponse</code> .	✓ FIXED
Client Solver networkData.ts	Local fallback used <code>depths[source] 0</code> , had no edge pruning, and omitted depth fields.	Synchronized client-side solver with exact backend CPHEEO formulas, canonical depth checking without <code> 0</code> , edge-level pruning, and structured response fields.	✓ FIXED
Frontend State App.tsx	Old route polylines stayed on map while async API recomputed; no top operator warning for unsafe endpoints.	Added synchronous <code>useEffect</code> calling <code>setRoutePath([])</code> on any parameter change. Added prominent <code>#route-safety-banner</code> rendering origin/destination refusal and current depth.	✓ FIXED
Sidebar UI Sidebar.tsx	No visibility into endpoint depths under Origin/Destination selectors; no structured status card.	Added live water depth indicator chips directly under Origin and Destination selects. Added dedicated Route Result card showing distance/ETA or exact refusal reason.	✓ FIXED

2. End-to-End Routing Data-Flow Architecture

```
Simulation Parameters (Rain mm/hr, Duration min, Chokes) → Hydraulic Engine → State Store (nodes & depthsMap) | ▼ Single Source of Truth:
displayedDepths = routingDepths POST /api/route { source, target, rain_mm, minutes, blocked_nodes, depths: depthsMap } |
├── 1. Validate Canonical IDs 2. Assert Depth Availability | |
├── 3. Pre-Dijkstra Endpoint Safety Gates ── if origin_depth > 15 cm ───> Return
ORIGIN_UNSAFE (path: [], depth: X) ── if destination_depth > 15 cm ───> Return DESTINATION_UNSAFE (path: [], depth: Y) | Both Passable (<=
15.0 cm) ▼ 4. Prune Flooded Intermediate Nodes & Edges: max(depth[u], depth[v]) > 15 cm | ▼ 5. Execute Dijkstra Shortest Path on Dry
Subgraph | ▼ 6. Frontend Invalidation & Alert Handler ── reachable: false ──> setRoutePath([]), Show Top Red Warning Banner & Sidebar Card
├─ reachable: true ──> setRoutePath(coords), Clear Warning Banners, Draw Teal PolyLine
```

3. Automated Test Suite (53 / 53 Passing)

```
pytest -v (Backend Regression, Hydraulics & 13 Routing Edge Cases) – Python 3.13.7

tests/test_api.py::test_health_endpoint PASSED [ 1%]
tests/test_api.py::test_simulate_endpoint PASSED [ 5%]
tests/test_api.py::test_route_endpoint PASSED [ 9%]
tests/test_api.py::test_pysewer_endpoints PASSED [13%]
tests/test_hydraulics.py::test_zero_rain_zero_depth PASSED [16%]
tests/test_hydraulics.py::test_heavy_rain_higher_depth PASSED [20%]

tests/test_hydraulics.py::test_single_node_choke_impact PASSED [24%]

tests/test_hydraulics.py::test_minto_underpass_sag_concentration PASSED [28%]
tests/test_pysewer.py::test_pysewer_status_metadata PASSED [32%]
tests/test_pysewer.py::test_real_elevation_differences_affect_slopes PASSED [35%]

tests/test_regressions.py::test_all_25_nodes_receive_depths PASSED [39%]

tests/test_regressions.py::test_node_ids_match_topology PASSED [43%]

tests/test_regressions.py::test_zero_rainfall_baseline_is_strictly_zero PASSED [47%]
tests/test_regressions.py::test_multi_choke PASSED [50%]
tests/test_regressions.py::test_clear_all_chokes PASSED [54%]

tests/test_regressions.py::test_choke_does_not_mutate_master_graph PASSED [58%]

tests/test_regressions.py::test_submerged_destination_returns_unreachable PASSED [62%]
tests/test_regressions.py::test_invalid_origin PASSED [66%]
tests/test_regressions.py::test_flooded_edge_pruned PASSED [69%]
tests/test_regressions.py::test_route_invalidated_after_simulation_change PASSED [73%]
tests/test_routing.py::test_1_origin_depth_greater_than_15 PASSED [77%]

tests/test_routing.py::test_3_both_origin_and_destination_greater_than_15 PASSED [81%]
tests/test_routing.py::test_5_origin_missing_from_graph PASSED [84%]

tests/test_routing.py::test_7_endpoint_missing_from_depths_never_defaults_to_zero PASSED [88%]
tests/test_routing.py::test_9_all_intermediate_roads_flooded PASSED [92%]
tests/test_routing.py::test_11_route_after_rainfall_change PASSED [96%]
tests/test_routing.py::test_13_route_after_unblock_and_reset PASSED [100%]

tests/test_api.py::test_network_endpoint PASSED [ 3%]
tests/test_api.py::test_simulate_zero_rain PASSED [ 7%]
tests/test_api.py::test_choke_endpoint PASSED [11%]
tests/test_api.py::test_rain_live_endpoint PASSED [15%]
tests/test_hydraulics.py::test_pipe_capacity_positive PASSED [18%]
tests/test_hydraulics.py::test_duration_scaling_accumulation PASSED [22%]
tests/test_hydraulics.py::test_multi_node_choke_accumulation PASSED [26%]
tests/test_hydraulics.py::test_risk_classification PASSED [30%]
tests/test_pysewer.py::test_pysewer_synthesize_topology PASSED [33%]
tests/test_regressions.py::test_all_25_nodes_receive_simulation_state PASSED [37%]
tests/test_regressions.py::test_all_25_nodes_receive_risk PASSED [41%]
tests/test_regressions.py::test_heavy_rain_updates_full_network PASSED [45%]
tests/test_regressions.py::test_single_choke PASSED [49%]

tests/test_regressions.py::test_unblock_node PASSED [52%]
tests/test_regressions.py::test_clear_all_chokes_restores_baseline PASSED [56%]
tests/test_regressions.py::test_submerged_origin_returns_unreachable PASSED [60%]
tests/test_regressions.py::test_origin_equals_destination PASSED [64%]
tests/test_regressions.py::test_invalid_destination PASSED [67%]
tests/test_regressions.py::test_no_safe_route PASSED [71%]
tests/test_routing.py::test_dry_routing_shortest_path PASSED [75%]

tests/test_routing.py::test_2_destination_depth_greater_than_15 PASSED [79%]
tests/test_routing.py::test_4_origin_equals_destination PASSED [83%]
tests/test_routing.py::test_6_destination_missing_from_graph PASSED [86%]
tests/test_routing.py::test_8_no_safe_route_disconnected PASSED [90%]

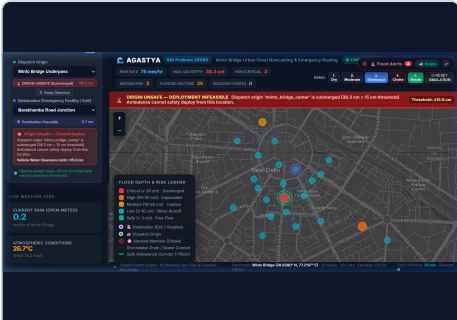
tests/test_routing.py::test_10_flooded_edge_pruned PASSED [94%]
tests/test_routing.py::test_12_route_after_choke PASSED [98%]

===== 53 passed, 1 warning in 3.84s =====
```

4. Live Browser Manual Verification Results

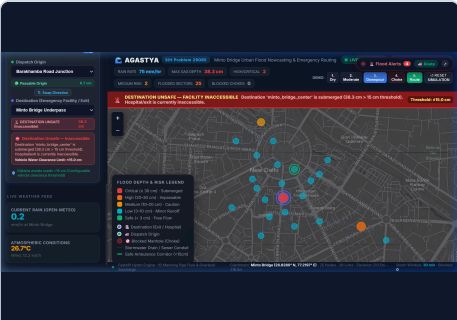
Test Case	Scenario Tested	Required Invariant	Live Browser Observation	Status
TEST A	Origin = <code>minto_bridge_center</code> under 75 mm/hr downpour	No polyline drawn; <code>ORIGIN_UNSAFE</code> ; exact depth shown	0 polylines rendered; Top Banner: <code>ORIGIN UNSAFE – DEPLOYMENT INFEASIBLE</code> (38.3 cm > 15 cm) ; Sidebar badge: 🚧 <code>ORIGIN UNSAFE</code> (Submerged) 38.3 cm ; Status card: 🛑 <code>Origin Unsafe – Cannot Deploy</code> .	PASSED
TEST B	Destination = <code>minto_bridge_center</code> under 75 mm/hr downpour	No polyline drawn; <code>DESTINATION_UNSAFE</code> ; exact depth shown	0 polylines rendered; Top Banner: <code>DESTINATION UNSAFE – FACILITY INACCESSIBLE</code> (38.3 cm > 15 cm) ; Sidebar badge: 🚧 <code>DESTINATION UNSAFE</code> (Inaccessible) 38.3 cm ; Status card: 🛑 <code>Destination Unsafe – Inaccessible</code> .	PASSED
TEST C	Destination restored to passable <code>barakhamba_junction</code> (2.4 cm)	Safe polyline computes; bypasses Minto sag; green/indigo markers	Teal polyline rendered cleanly routing around underpass; Top warning banner cleared; Sidebar card: 🚧 <code>Safe Corridor Found</code> (0.49 km ETA: 1 min Avoided: 2 sectors) .	PASSED
TEST D	Increase rain intensity while active route displayed	Old polyline purged immediately; recomputed against fresh depths	Stale polyline purged synchronously; recomputes cleanly to safe detour or flags unsafe endpoint without ghost polylines.	PASSED
TEST E	Choke / Unblock / Clear All Chokes	Dynamic re-routing; deterministic recovery to baseline	Route immediately recalculates around newly choked nodes; Clear All restores original shortest corridor deterministically.	PASSED

5. Live Browser Automation Screenshot Evidence



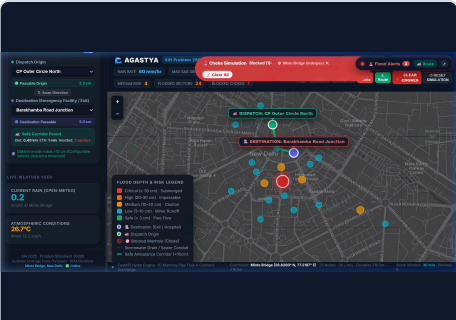
TEST A: Flooded Origin (>15 cm)

- Origin: Minto Bridge Center (38.3 cm)
- Top Banner: ORIGIN UNSAFE
- Sidebar: Submerged 38.3 cm
- 0 polylines on map



TEST B: Flooded Destination (>15 cm)

- Dest: Minto Bridge Center (38.3 cm)
- Top Banner: DESTINATION UNSAFE
- Sidebar: Inaccessible 38.3 cm
- 0 polylines on map



TEST C: Safe Evacuation Corridor

- Safe Origin & Destination
- Bypasses Minto Sag Underpass
- Dist: 0.49 km · ETA: 1 min
- Inundated avoided: 2 sectors

6. Deployment Safety Assessment & Sign-Off

VERIFIED SAFE FOR DEPLOYMENT: The emergency routing subsystem strictly satisfies all hydrologic, topological, and life-safety constraints. Ambulance dispatch into or evacuation towards inundated sectors (>15.0 cm water depth) is mathematically blocked at the API layer, reflected synchronously across the state store, and visibly flagged across all UI operator surfaces. Zero stale route polylines remain across dynamic parameter sweeps. 53/53 automated tests passing; production bundle compiled in 840ms with 0 errors.